LatchMoE: Graph-Compatible Expert Offloading through Address-Stable Residency

Anonymous submission

Abstract

Sparse mixture-of-experts (MoE) models reduce arithmetic per token but retain a large parameter footprint. Expert offloading can fit such models on a memory-constrained accelerator, yet conventional offloading changes weight locations on the decode path. This behavior conflicts with graph replay, which relies on stable addresses and a repeatable execution topology; disabling replay exposes a host-bound decode path.

We present LatchMoE, an expert-offloading runtime that separates dynamic expert residency from captured tensor addresses. LatchMoE preallocates fixed device slots, stages routed experts outside the replay boundary, protects slots with an explicit transfer/compute lifecycle, and executes oversized prefill working sets in capacity-bounded waves. The resulting data plane accepts placement plans without embedding a particular prediction policy.

The implementation targets a hook-enabled vLLM-Ascend stack. A graph-only qualification on Qwen3-30B-A3B records exact output agreement, stable addresses, transfer ordering, bounded residency, and service release across three independent starts. In three matched pairs, capacity-bounded multi-wave prefill reduces repeat-median TTFT p50 and p95 by 35.21% and 22.96%, respectively, relative to graph-compatible full-layer staging. We bound this result to one model, device, memory configuration, and low-concurrency workload.

1 Introduction

Mixture-of-Experts (MoE) architectures have emerged as a prominent approach for scaling Large Language Models (LLMs) to hundreds of billions of parameters [2, 5, 7, 13]. By dynamically routing each token to only a sparse subset of experts, MoE models substantially increase model capacity while keeping per-token computation bounded [3, 12]. However, sparse activation reduces computation rather than model storage: although only a small number of experts are activated for each token, the runtime must still make the entire expert pool available for routing. As MoE models scale, expert weights can therefore dominate the memory footprint and impose substantial pressure on accelerator High-Bandwidth Memory (HBM). To serve such models on memory-constrained accelerators, **expert offloading** has become an important strategy: the runtime keeps non-expert layers and a bounded subset of frequently used experts in local HBM, while fetching non-resident experts from host memory on demand [6, 10, 18]. Expert offloading makes otherwise memory-infeasible MoE models deployable on limited-HBM devices, but introduces dynamic expert movement into the inference critical path.

To make expert offloading practical, prior systems reduce its data-movement overhead through expert caching, routing-aware prediction and prefetching, and transfer–computation overlap [1,4,6,8,18]. These techniques reduce the frequency of host-to-device transfers or hide part of their latency, substantially improving the efficiency of offloaded MoE inference. However, they do not eliminate the inherent execution dynamism introduced by expert offloading. Routing-driven cache misses can still trigger expert loading, eviction, and replacement, causing the set of HBM-resident experts to evolve across decoding steps. More importantly, loading and replacing experts in a bounded HBM cache can change the mapping between logical experts and their physical weight locations across decoding steps. Thus, existing data-movement optimizations improve which experts are moved and when they are moved, but they do not eliminate runtime changes in expert residency or logical-to-physical weight bindings.

Such runtime changes can be naturally accommodated under eager execution, where the host resolves the current expert placement and dispatches the corresponding kernels at each decoding step. This flexibility, however, comes at a substantial performance cost. Autoregressive decoding repeatedly executes a largely recurring sequence of fine-grained accelerator kernels, and launching these kernels individually incurs repeated host-side scheduling, dispatch, and synchro-

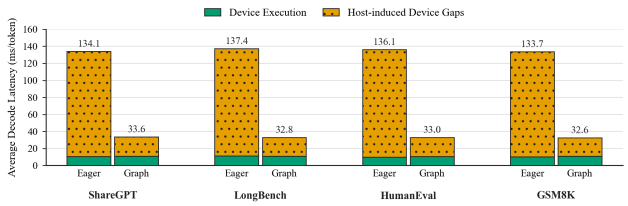

Figure 1: Average decode latency breakdown under eager execution and graph replay. Graph replay substantially reduces host-induced device gaps while leaving device execution time nearly unchanged.

nization overhead. **Graph replay** captures the recurring execution sequence once and replays it as a unit [19].

As shown in Figure 1, graph replay reduces average decode latency by approximately 75% across four workloads, from 133.7–137.4 ms/token under eager execution to 32.6–33.6 ms/token, while device execution time remains nearly unchanged. This result indicates that most of the improvement comes from eliminating host-induced device idle time rather than accelerating the underlying kernels. Therefore, falling back to eager execution is not merely a compatibility inconvenience; it forfeits a major source of decoding efficiency.

However, these efficiency gains rely on a **replay-stable execution state**, in which the captured execution structure and graph-visible memory bindings remain valid across replays. Dynamic expert offloading creates a fundamental tension with this requirement. When routing-driven cache misses trigger expert replacement, the same logical expert may be served from different physical weight locations across decoding steps, changing the logical-to-physical binding observed by captured computation. Such binding changes prevent the runtime from safely reusing the captured execution state unless it repairs the affected graph state or reintroduces eager intervention, both of which erode the host-overhead savings of graph replay. The central challenge is thus to **preserve graph replay while allowing expert residency and physical placement to evolve dynamically under a bounded HBM budget**.

Existing systems address the two sides of this tension largely in isolation. Expert-offloading systems optimize *what changes* at runtime—which experts remain resident, which experts are transferred, and when those transfers occur—through caching, prediction, prefetching, and transfer–computation overlap [1, 4, 6, 8]. These techniques improve expert availability under constrained memory, but dynamic replacement and the resulting logical-to-physical bindings remain part of the runtime-managed execution state. In contrast, graph-compatible inference runtimes focus on *what must remain reusable*, stabilizing graph-visible workspaces, metadata, and execution configurations so that captured computation can be replayed efficiently [19]. However, such mechanisms do not virtualize a dynamically replaceable expert working set within a bounded HBM budget. Consequently, neither line of work provides an execution abstraction that simultaneously hides dynamic expert replacement from captured computation and preserves efficient graph replay.

Our key insight is that **graph replay requires stable physical storage, not static expert residency**. A captured MoE computation does not require the same logical expert to remain at the same physical location across decoding steps; it only requires the storage referenced by the captured computation to remain valid for replay. This allows us to preallocate a fixed set of physical expert slots and reuse them for different logical experts as routing changes. Before the corresponding expert computation is replayed, the runtime places the required experts into these slots and updates the logical-to-slot mapping, while the captured computation continues to access the same physical storage. In this way, expert replacement changes *what is stored* in the replay-visible slots rather than *where the captured computation accesses*, allowing dynamic expert offloading to coexist with graph replay under a bounded HBM budget.

Realizing this design raises three systems challenges. **First, the fixed slot interface must support runtime expert staging without invalidating graph replay.** The active experts are known only after routing, while expert loading and replacement depend on this runtime information. These dynamic operations must therefore occur outside captured computation without changing the tensors, addresses, or execution structure expected by subsequent replayed MoE computation. **Second, the fixed slot set must be updated efficiently as routing changes, without turning expert transfers into serial stalls.** A naive runtime can stage all missing experts before starting expert computation, placing host-to-device transfer latency directly on the inference critical path. This problem becomes more pronounced when the routed working set exceeds the available slots, because the required experts cannot be materialized simultaneously. The runtime must therefore update the bounded slot set while retaining opportunities to overlap staging for upcoming experts with computation on experts that are already ready. **Third, slot reuse must remain correct under asynchronous transfer and computation.** A slot cannot be overwritten while its current expert is still being consumed by in-flight computation, and a newly assigned expert cannot become visible before its weight transfer has completed.

We present LatchMoE, a graph-compatible MoE offloading system that virtualizes dynamic logical experts over a fixed physical slot interface. **First, LatchMoE isolates routing-dependent expert staging at an explicit eager boundary between a captured router and captured expert computation.** Each offloaded layer preallocates a fixed slot bank and a persistent logical-to-slot mapping before

graph capture. At replay time, the staging boundary loads the required experts and updates the mapping in place, while the downstream captured computation continues to access the same slot and mapping addresses. **Second, LatchMoE pipelines expert staging and computation through wave-based execution.** Each wave stages only a subset of the routed experts into the fixed slot space, allowing the same physical storage to be reused as the active expert set changes. By staging upcoming experts while already prepared experts are being executed, LatchMoE overlaps host-to-device transfers with expert computation and reduces transfer-induced device bubbles. **Third, LatchMoE coordinates this overlapped execution through a version-controlled slot lifecycle.** Explicit slot ownership, versioning, and transfer–compute dependencies ensure that a slot is neither overwritten while its current contents are still in use nor exposed to computation before newly staged weights are ready. Together, these mechanisms preserve a stable graph-visible execution interface while allowing expert residency to evolve dynamically and efficiently.

In summary, this paper makes the following contributions:

- We identify and characterize a fundamental tension between dynamic expert offloading and graph replay in memory-constrained MoE inference. Routing-driven expert replacement changes logical-to-physical weight bindings, whereas efficient graph replay relies on a stable graph-visible execution state. This observation shows that replayability requires stable physical storage rather than static expert residency.
- We introduce a replay-stable expert-slot abstraction that decouples dynamic logical expert placement from the physical storage exposed to captured computation. By virtualizing logical experts over preallocated physical slots and persistent mappings, LatchMoE allows expert residency to change at runtime while preserving the graph-visible addresses required for replay.
- We design a graph-compatible execution framework that combines routing-dependent expert staging with pipelined and concurrency-safe slot reuse. LatchMoE isolates dynamic staging from captured computation, overlaps expert transfer with computation through wave-based execution, and coordinates slot ownership and readiness to preserve correctness under asynchronous execution.
- We implement LatchMoE on top of vLLM-Ascend and evaluate it with representative MoE workloads under constrained HBM. Within a matched single-device configuration, LatchMoE preserves exact outputs and graph replay under dynamic expert offloading, while capacity-bounded multi-wave prefill reduces repeat-median TTFT p50 by 35.21% relative to graph-compatible full-layer staging.

2 Background and Problem Formulation

2.1 Sparse activation and expert residency

An MoE layer contains a set of expert MLPs and a router that assigns each token to a small subset of them. Sparse routing limits the expert computation performed for one token, but it does not remove the requirement that any expert selected by the router must be available to the runtime [5, 13]. Expert-offloading systems therefore keep a bounded resident set in accelerator memory and retain the remaining expert weights in host memory. A cache hit can proceed from the resident copy, whereas a miss requires host-to-device transfer and may require evicting a resident expert. Prior systems reduce this cost through caching, prediction, prefetching, and transfer–computation overlap [1, 4, 6, 8, 18].

This data path has two forms of state. The *logical state* records which experts are currently resident. The *physical state* records the device allocations and indices through which kernels access their weights. Eager execution can resolve both forms before every kernel launch. Expert eviction can change a pointer or an index because the host constructs the next launch from the current residency state. This flexibility also places routing, placement, launch, and synchronization decisions on the host path.

2.2 Graph replay and the binding conflict

Accelerator graph replay captures a repeated operation sequence and reissues it with reusable execution state. Nimble compiles dynamic neural-network execution, while FlashInfer applies graph capture and replay to LLM inference [14, 19]. The graph breakdown in Figure 1 motivates why LatchMoE treats replay as a requirement rather than an optional optimization: in the measured workloads, eager execution spends most decode time in host-induced gaps.

Dynamic expert replacement conflicts with replay when captured computation observes the placement decision through changing tensor bindings. Reallocating an expert or changing the weight object passed to the fused MoE operation makes the next iteration differ from the captured state. Moving the entire MoE path back to eager execution restores dynamic placement but also restores the host overhead that replay removes. Keeping all experts resident avoids replacement but defeats offloading under a constrained HBM budget.

The conflict is narrower than requiring static residency. Captured computation needs stable graph-visible storage and metadata addresses. The logical expert stored at each address may change between replays if three conditions hold: the required bytes arrive before they are consumed, the mapping names the new owner only after that transfer is ready, and no slot is reused while an older owner is still in flight. This distinction between stable storage and dynamic contents is

the basis of LatchMoE.

2.3 Requirements

We derive five requirements for a graph-compatible offloading data plane.

1. **Stable graph-visible allocations.** The expert-weight tensors and mapping buffers referenced by captured computation retain their addresses across replay.

2. **Explicit dynamic boundary.** Routing-dependent placement and weight transfer execute outside captured regions without changing the model's routing decisions.

3. **Safe publication and reuse.** A mapping becomes visible only after the corresponding transfer is ordered before its consumer, and a slot cannot be reassigned while compute still holds its current version.

4. **Bounded exact execution.** If a routed working set exceeds slot capacity, execution remains within the configured storage bound without dropping or approximating routed token–expert pairs.

5. **Fail-closed observability.** Capture, replay, fallback, transfers, ownership, memory, and service release remain observable. An eager fallback cannot be reported as a successful graph-compatible run.

3 Design

3.1 Overview

LatchMoE separates replay-stable storage from routing-dependent state changes. Figure 2 shows the resulting execution model. At initialization, a CPU-first host store retains expert weights and LatchMoE allocates fixed device slot banks and persistent logical-to-slot mapping buffers. During inference, a captured router produces the routed expert IDs. An eager staging boundary turns those IDs into a placement plan, transfers missing weights, and updates mapping values in place. A captured expert MLP then consumes the same slot and mapping allocations on every replay.

The design has three core mechanisms. Replay-stable slot virtualization decouples expert identity from physical allocation. Replay-boundary staging publishes a new mapping only after transfer readiness. A versioned lifecycle and a capacity-bounded wave executor make reuse correct when transfer and compute overlap or when the active set exceeds capacity. The placement-plan interface orders routed experts but does not embed a prediction policy in the data plane.

3.2 Replay-stable expert-slot virtualization

For each managed MoE layer, LatchMoE allocates two contiguous tensors that hold the layer's fixed set of physical W_{13} and W_2 expert slots. The first dimension indexes physical slots; each slice has the shape and layout expected by the fused expert kernel. These allocations persist across capture and replay. A second persistent tensor maps each logical expert ID to a physical slot ID. Captured computation receives the slot tensors and mapping buffer, so its graph-visible pointers do not depend on which logical experts are resident.

Each slot records a logical owner, a monotonically increasing version, a state, and its last-use step. Reassigning a slot removes the old logical owner from the resident index, installs the new owner, increments the version, and enters the loading state. A lease is the tuple of slot ID, logical expert, and version. The version distinguishes two ownership intervals that reuse the same physical slot and lets asynchronous completions reject stale work.

The mapping is valid only for ready experts. Before the captured MLP consumes routed IDs, LatchMoE remaps them through the persistent logical-to-slot tensor. The physical expert count remains the number of allocated slots rather than the number of logical experts. Thus replacement changes slot contents and mapping values, while the tensors bound into the captured computation remain fixed.

3.3 Replay-boundary staging and safe publication

The router must run before the runtime knows the active expert set, but the expert MLP must run after all selected weights are available. LatchMoE exposes the interval between these operations as an eager staging boundary. The captured router produces routed expert IDs, the runtime deduplicates that set, and a placement plan orders the routed experts. The plan must be a permutation of the actual routed set: it may change staging order, but it cannot add an unrouted expert or omit a routed expert.

For a cache miss, the transfer engine assigns a slot version, enqueues the expert's W_{13} and W_2 copies on a dedicated H2D stream, and records a readiness event. Publication follows a fixed order. The runtime first validates that the slot still matches the lease, installs the readiness event as a dependency of the consumer stream, changes the slot to ready, and only then publishes the logical-to-slot mapping. This order prevents captured compute from observing either incomplete weights or a completion event that belongs to an older slot owner.

Hits and misses share the same output interface: ready physical slot IDs plus the persistent mapping tensor. The captured MLP therefore does not encode whether an expert was already resident, loaded on demand, or prefetched for an

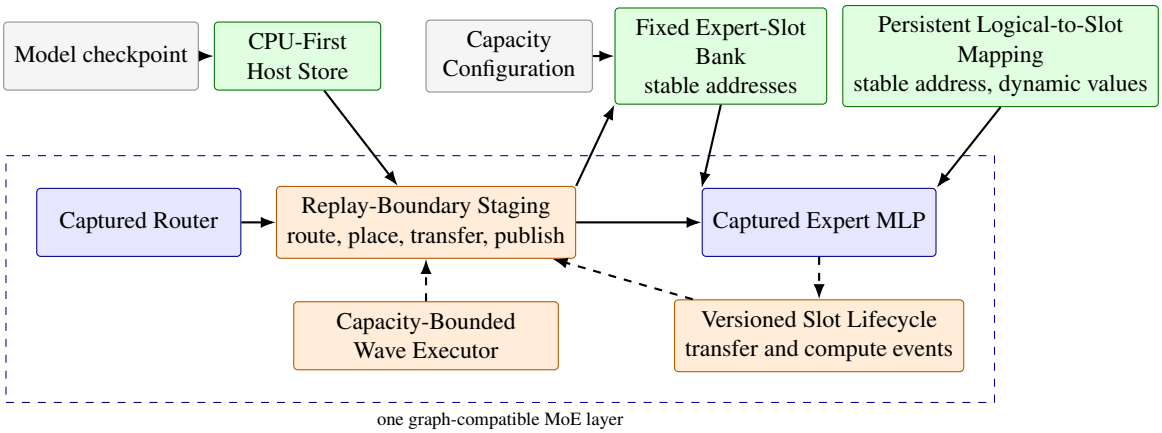

Figure 2: LatchMoE keeps graph-visible slot and mapping addresses fixed while changing their contents and values at an eager replay boundary. Solid arrows carry weights or routing data; dashed arrows carry wave and lifecycle control. Blue blocks are captured, orange blocks execute eagerly, and green blocks are stable allocations.

upcoming wave.

3.4 Versioned transfer–compute lifecycle

The slot state machine contains four externally relevant states: empty, loading, ready, and computing. Empty and ready slots may be selected for a new owner, while loading and computing slots are not reassignable. Before expert compute begins, the runtime acquires leases for the active slots and marks them computing. A completion event protects those leases until the consumer stream has finished. Release validates that owner and version still match the recorded lease before returning a slot to the ready state.

This protocol enforces two safety properties. First, transfer completion cannot make a stale owner ready because readiness checks the version created when the copy was issued. Second, eviction cannot overwrite an in-flight expert because a computing slot is excluded from reassignment. Violations raise an error instead of redirecting execution to eager mode. The fail-closed behavior makes graph compatibility part of the runtime contract rather than an unobserved best effort.

3.5 Capacity-bounded multi-wave execution

Decode commonly activates a small expert set, but a prefill request aggregates routing decisions over many tokens. The union of routed experts can exceed the slot capacity even though each token selects only a few experts. LatchMoE handles this overflow by partitioning routed token–expert pairs into waves. Each wave contains no more distinct missing experts than its staging bank can hold. Hit-only work may be scheduled separately so that a ready main-slot expert need not be copied into a temporary wave buffer.

Wave execution preserves the router's assignments. LatchMoE retains every routed pair's original flattened top-k position, computes wave-local expert outputs, concatenates those outputs, reconstructs one global native unpermute index, and invokes the ordinary token-combine operation once with the original top-k weights and output shape. It rejects incomplete or duplicate pair coverage. The single final combine avoids the numerical reordering introduced by independently combining and accumulating each wave.

Multiple stage banks form a software pipeline. While the current wave computes, the transfer stream may populate another bank for the next wave. The scheduler can order waves by their required transfer bytes, but it cannot change pair membership or final combination semantics. A stage bank is reused only after its compute lease completes, so the same versioned lifecycle protects both the persistent decode slots and transient prefill slots.

3.6 Policy-neutral placement

LatchMoE defines the data-plane constraints for a placement plan: target layer, complete routed set, and a valid ordering. The default behavior preserves the router's first-seen order. An external controller may instead prioritize predicted reuse or transfer cost, but it must return a permutation of the routed experts and obey the same capacity and lifecycle checks. Separating this interface from slot management allows placement policies to share one graph-compatible execution substrate and keeps policy quality outside the correctness argument of this paper.

4 Implementation

We implement LatchMoE as a Python package that registers through vLLM's general plugin entry point. The package integrates with a hook-enabled vLLM-Ascend runtime; the upstream request scheduler and OpenAI-compatible serving API remain unchanged. The host runtime and the Ascend platform plugin are pinned as a compatible stack because the staging boundary changes the internal fused-MoE execution contract rather than the public serving interface.

4.1 Graph integration

The integration decomposes a managed MoE layer into a captured router, the custom `vllm::moe_offload_stage` splitting operation, and a captured expert MLP. The splitting operation returns control to the host after routing so that LatchMoE can stage the active experts. The subsequent graph piece reads the persistent slot weights and logical-to-slot buffer. The runtime locks their data pointers at capture and validates them before replay.

Graph-compatible runs use PIECEWISE ACLGraph. The benchmark validator requires explicit capture and replay records and rejects forced eager execution, eager fallback, graph errors, address changes, stale H2D completion, and slot-version violations. Disabling LatchMoE restores the host runtime's null hooks. The launcher also rejects simultaneous use of the native weight-offload path and LatchMoE because two independent owners of expert storage would violate the slot lifecycle.

4.2 Loading and memory management

CPU-first loading creates managed expert tensors in host memory during model initialization. The host store keeps one contiguous W_{13} tensor and one contiguous W_2 tensor per layer and exposes per-expert views to the transfer engine. It checks shape, dtype, stride, device, layer coverage, and expert coverage before releasing the original device expert banks. Host pinning is used when the runtime supports it and reports failures explicitly.

Automatic configuration derives the resident-layer selection and slot count from the requested offload amount, physical HBM, a KV-cache reserve, and a cap on the fraction of HBM assigned to slot storage. Explicit overrides remain available for controlled sensitivity experiments. The memory ledger accounts separately for the host store, persistent slots, transient prefill stage banks, mapping tensors, full-layer fallback storage, and any original weights that remain allocated.

4.3 Transfers and lifecycle enforcement

The transfer engine maintains a dedicated H2D stream. It validates layout compatibility before copying and batches consecutive host and slot slices when their storage is contiguous. Each asynchronous load returns a readiness object that contains the NPU event and the exact slot leases made consumable by that event. The consumer installs the event dependency before the object marks the leases ready. The runtime then publishes mapping values.

Compute protection uses an event recorded after the expert operation. Slots remain in the computing state until the event completes and every recorded lease still matches its current owner and version. Shutdown drains pending transfer and compute work, closes profiling writers, releases managed storage, and emits a release acknowledgement for the benchmark harness.

4.4 Wave execution and profiling

The prefill path supports a configurable number of stage banks and prefetch depth. The qualified configuration uses two stage banks and depth one. Wave plans carry the original pair offsets needed for one native recombination, and strict validation rejects duplicate or missing coverage. A recoverable preflight or native-recombination qualification failure may select the explicit full-layer overflow mode; NPU, ACL, memory, address, and lifecycle failures are not swallowed by that path.

Profiling records JSONL events for layer registration, slot ownership, transfers, mapping publication, wave staging and compute, graph replay issue, memory accounting, and service release. Benchmark manifests bind each run to the runtime revisions, model and workload digests, device, command line, environment, graph mode, and raw client/server artifacts. These records support the fail-closed checks used in Section 5.

5 Evaluation

Our current evaluation answers three questions within a deliberately narrow qualification boundary:

Q1: Replay and correctness. Does LatchMoE preserve graph-visible addresses, transfer ordering, capacity bounds, and exact model outputs across independent service starts?

Q2: Multi-wave staging. Under a matched graph-compatible contract, how does capacity-bounded multi-wave prefill compare with blocking full-layer staging?

Q3: Mechanism exercise. Does the online runtime issue next-wave transfers before current-wave compute, pre-

Table 1: LatchMoE preserves exact outputs and graph/lifecycle invariants across three independent service starts. TTFT and TPOT are qualification measurements, not a matched performance comparison.

Metric	Run 1	Run 2	Run 3
Successful requests	11/11	11/11	11/11
Exact output tokens	1,408	1,408	1,408
TTFT p50 (ms)	573.64	537.70	573.11
TPOT p50 (ms/token)	55.33	55.93	53.56
Peak/final HBM (%)	91/5	91/5	91/5
Graph fallback events	0	0	0

serve native recombination, and account for its bounded storage?

These questions validate the runtime mechanism and one matched prefill result. They do not establish performance breadth across models, devices, memory budgets, workloads, or serving concurrency.

5.1 Experimental setup

All reported graph-only experiments use Qwen3-30B-A3B in BF16 on one physical Ascend 910B2 with tensor parallelism one. LatchMoE manages 12 MoE layers with a 14-GiB host-offload target and 32 physical slots. The server uses `max_num_seqs=1`; client concurrency is one and prefix caching is disabled. Requests are drawn from a fixed ShareGPT manifest. The matched multi-wave experiment uses the same ordered 200-request manifest in every unit and generates 128 output tokens per request.

Every formal unit starts a fresh managed service, requires PIECEWISE ACLGraph capture and replay, retains raw client and server artifacts, and ends with a release acknowledgement and a physical HBM sample. Runs with eager fallback, forced eager execution, NPU/ACL errors, OOM, address changes, stale transfer events, or slot-generation violations fail validation. Exact output comparison uses request IDs and complete output-token arrays, not decoded string equality.

5.2 Q1: Replay-stable execution and exactness

Table 1 summarizes three independent formal starts from the graph-lifecycle qualification. Each start serves 11 requests and 1,408 output tokens. The first repeat is compared with the preceding short gate, and later repeats are compared with the prior repeat. Across the three formal starts, all 33 requests and all 4,224 output tokens match exactly.

The runtime-level gates also pass in every start. All 12 managed layers retain the slot-bank and logical-map pointers recorded at capture. Every checked compute lease preserves its owner and version until completion. Every recorded decode miss installs the consumer dependency and reaches readiness before the logical mapping is published. No multi-wave active set exceeds the 32-slot capacity, and every service emits a release acknowledgement before physical HBM returns to 5%. These observations establish the replay and lifecycle contract for the evaluated configuration.

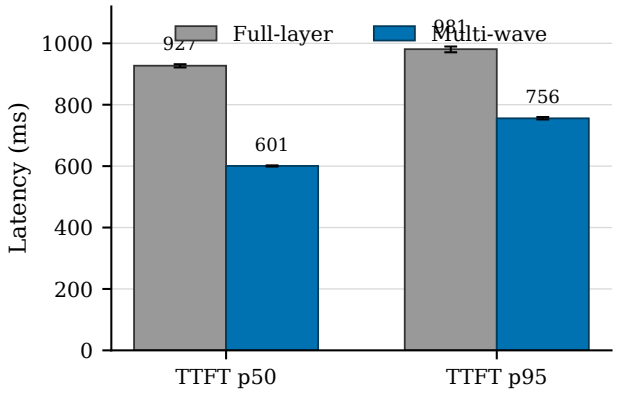

Figure 3: Capacity-bounded multi-wave staging reduces the repeat-median TTFT p50 by 35.21% and TTFT p95 by 22.96% relative to graph-compatible full-layer staging. Bars show the median of three independent service-start summaries; whiskers show their minimum and maximum. The comparison uses one Ascend 910B2, Qwen3-30B-A3B BF16, a 14-GiB offload target, 32 slots, and concurrency one.

Table 2: Matched full-layer and multi-wave results. Each arm aggregates three fresh starts and 600 ShareGPT requests. Latency entries are the median of the three repeat-level quantiles.

Mode	TTFT p50 (ms)	TTFT p95 (ms)	TPOT p50 (ms/token)
Full-layer	927.06	980.89	54.33
Multi-wave	600.65	755.68	54.42

5.3 Q2: Matched multi-wave prefill

We compare *full-layer* staging, which materializes the layer's expert working set before compute, with *multi-wave* staging, which executes the same routed work through bounded stage banks. The two arms differ only in the overflow-mode setting. We use a fixed AB/BA/AB order to form three matched pairs, with one fresh service start per unit.

As Figure 3 and Table 2 show, the repeat-median TTFT p50 falls from 927.06 to 600.65 ms, a 35.21% reduction. The corresponding TTFT p95 falls from 980.89 to 755.68 ms,

a 22.96% reduction. Pair-level p50 reductions range from 35.02% to 35.55%, and pair-level p95 reductions range from 21.71% to 23.63%.

TPOT does not show the same reduction. Its repeat-level p50 values are 53.87/54.74/54.33 ms/token for full-layer and 55.34/54.42/53.87 ms/token for multi-wave. The supported performance conclusion is therefore limited to time to first token in this prefill-heavy boundary, not a decode-speedup claim. Each arm completes 600/600 requests and 76,800 output tokens. All six units produce identical token arrays, capture and replay the graph, record zero fallback and runtime error events, release their managed service, and return to 5% final HBM.

5.4 Q3: Overlap, recombination, and memory bounds

The multi-wave stability profile contains 2,544 prefill-layer events and 9,077 waves. Every wave is issued before its compute begins. The runtime submits 6,533 next-wave prefetches before the current wave's compute and records no late after-compute prefetch issue. Aggregate stage wait is 1,593.87 ms across the profile, compared with 7,493.41 ms of profiled MLP time, and the maximum per-layer stage wait is 1.27 ms. These measurements show that the intended software-pipeline schedule executes online; they do not isolate the causal latency contribution of overlap.

Exactness is checked separately from scheduling. Eleven trigger requests match the full-layer oracle for serial eager multi-wave, overlapped eager multi-wave, and overlapped graph multi-wave execution, covering 1,408 token IDs per mode. A tensor-level check over 136 BF16 routed pairs split across five interleaved waves is bitwise equal to the native combine. The only online combine mode in the qualified profile is the native-recombination path.

The final memory ledger reports 13.50 GiB in the host expert store, 3.38 GiB in persistent device slots, and 0.56 GiB in two prefill stage banks. Total device slot and stage storage is 3.94 GiB. All 12 original expert-weight banks are released, no wave exceeds 32 slots, and the physical 65,536-MiB device peaks at 91% HBM without an OOM or memory-growth failure.

6 Discussion and Limitations

6.1 Stable storage is the reusable interface

LatchMoE's main lesson is that dynamic object identity and graph replay are not inherently incompatible. The conflict arises when identity changes are exposed as changes to graph-visible allocations or execution structure. A fixed storage interface moves that dynamism into contents and metadata that can be updated at a controlled boundary. The same principle may apply to other captured operators with a bounded, replaceable working set, provided that the runtime can identify a legal staging boundary and enforce publication and reuse ordering. Our experiments establish this lesson only for the implemented MoE path.

The versioned lease is equally important as the fixed allocation. Stable pointers alone do not prevent a transfer completion for an old owner from publishing stale contents, nor do they prevent eviction during in-flight compute. Address stability supplies replay compatibility; owner/version checks and stream dependencies supply temporal correctness. Treating these as one contract makes failures observable and testable.

6.2 Control policy is outside the data plane

LatchMoE accepts an ordered placement plan but constrains it to the actual routed expert set. This boundary lets future caching or prediction policies reuse the fixed slots, transfer engine, wave executor, and graph checks. The current evaluation does not compare placement policies and should not be read as evidence that the default ordering is optimal. Such a study requires a shared trace, identical storage capacity, and policy-specific hit, transfer, and latency measurements on the same data plane.

6.3 Scope of the current evidence

The online evidence covers one unquantized model, one Ascend 910B2, tensor parallelism one, a 14-GiB offload target, 32 slots, `max_num_seqs=1`, client concurrency one, and disabled prefix caching. It validates exactness, graph replay, lifecycle ordering, bounded residency, and service release in that configuration. The matched result further shows lower TTFT for multi-wave than full-layer staging under the same narrow contract. It does not show a general TPOT improvement.

Several dimensions remain open. Higher concurrency changes the union of active experts and may alter both slot pressure and transfer–compute overlap. A different model can change expert shapes, routing distributions, and the number of managed layers. Other slot counts and memory budgets can change wave count and KV-cache headroom. Prefix-cache reuse is disabled because the current correctness contract does not cover cache hits across the offload staging boundary. Multi-device expert parallelism introduces device-to-device communication and is not represented by the single-device host-to-device path.

Before making a broad serving claim, the evaluation must compare full residency, native prefetch offload, the legacy layered path, and LatchMoE under the same graph-only manifests. It must also add model, capacity, workload, and concurrency sensitivity, retain graph or capacity failures as results, and report dispersion across independent starts. Until that matrix is complete, the paper's evidence supports

a graph-compatible mechanism and the scoped multi-wave TTFT result.

7 Related Work

Sparse MoE inference. Sparsely gated MoE models increase parameter capacity while activating a subset of experts for each token [3, 5, 12, 13]. Systems for large MoE models also combine routing with model and expert parallelism [11,12]. LatchMoE does not change the model, router, or expert computation. It addresses the runtime storage interface used when expert weights cannot remain fully resident on one accelerator.

Expert offloading and scheduling. Expert-offloading systems reduce host-to-device movement or its exposed cost through caching, prediction, prefetching, mixed precision, fine-grained placement, and pipelined execution [1, 4, 6, 8, 10, 16–18]. FlexGen studies coordinated placement and execution across heterogeneous memory for dense-model inference [15]. These directions decide which weights to retain or move and when to move them. LatchMoE is complementary: its placement-plan interface can accept such decisions, while its contribution is the fixed physical slot, persistent mapping, and lifecycle abstraction that keeps replacement compatible with captured MoE execution.

Graph-based inference execution. Nimble compiles dynamic neural-network execution, while FlashInfer provides graph-oriented mechanisms for customizable LLM inference [14, 19]. vLLM reduces serving overhead through continuous batching and paged KV-cache management [9]. LatchMoE focuses on a different mutable resource: expert weights. It inserts an explicit eager boundary between captured routing and captured expert computation, then keeps the downstream weight and mapping allocations stable while their contents change.

Positioning. Prior offloading work primarily optimizes the dynamic placement decision; graph-oriented runtimes primarily optimize reuse of repeated execution. LatchMoE connects these concerns by defining which state must remain stable and which state may change between captured regions. It leaves policy quality to the former line of work and preserves the replay interface required by the latter.

8 Conclusion

LatchMoE decouples dynamic logical expert residency from the stable physical storage required by graph replay. Fixed expert slots and persistent mappings form the replay-visible interface; an eager staging boundary updates their contents; versioned transfer and compute dependencies make reuse safe; and capacity-bounded waves execute oversized prefill working sets with one native recombination. In the qualified single-NPU configuration, three independent starts preserve exact outputs, stable addresses, transfer ordering, bounded residency, graph replay, and resource release. A matched six-unit experiment shows that multi-wave staging reduces repeat-median TTFT p50 by 35.21% and TTFT p95 by 22.96% relative to full-layer staging, while TPOT remains comparable. These results support the replay-stable storage abstraction and its scoped prefill benefit; broader serving claims require the remaining baseline and sensitivity matrix.

References

[1] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[2] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[3] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5547–5569, 2022.

[4] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 574–588, 2025.

[5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[6] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pregated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, pages 1018–1031, 2024.

[7] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[8] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6710–6720. Association for Computational Linguistics, 2024.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[10] Han Li, Jingwei Sun, Junqing Lin, and Guangzhong Sun. CommitMoE: Efficient fallback-free MoE inference with offloading under GPU memory constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 22904–22912, 2026.

[11] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, 2023.

[12] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18332–18346, 2022.

[13] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

[14] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. Nimble: Efficiently compiling dynamic neural networks for model inference. In *Proceedings of Machine Learning and Systems*, 2021.

[15] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[16] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.

[17] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.

[18] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh K. Marina. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.

[19] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie W. Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *arXiv preprint arXiv:2501.01005*, 2025.